Application of Graphs for Story Generation in Video Games

Iwona Grabska-Gradzińska
Department of Games Technology, Faculty of Physics, Astronomy and Applied Computer Science, Jagiellonian University, Krakow, Poland
iwona.grabska@uj.edu.pl

Leszek Nowak
Department of Information Technologies, Faculty of Physics, Astronomy and Applied Computer Science, Jagiellonian University, Krakow, Poland
leszek.nowak@uj.edu.pl

Wojciech Palacz
Department of Design and Computer Graphics, Faculty of Physics, Astronomy and Applied Computer Science, Jagiellonian University, Krakow, Poland
wojciech.palacz@uj.edu.pl

Ewa Grabska
Department of Design and Computer Graphics, Faculty of Physics, Astronomy and Applied Computer Science, Jagiellonian University, Krakow, Poland
ewa.grabska@uj.edu.pl

ABSTRACT

The paper presents the system based on the layered hierarchical graphs for game plot description implemented in Godot Engine using the JSON format. The aim of the presented research is to create a tool that both supports the decision-making process and helps multiple writers to collaborate in creating a game world. Constructing a layered hierarchical graph provides information about game plot description, while the proposed graph generation system based on JSON data format allows designers and story writers to create game worlds. A key feature of the graph generation system is that it describes game worlds using a sequence of generation rules which represent the player's actions. On the one hand, the proposed approach helps the story writers to control the impact of the created story elements on the game world. On the other hand, it allows world designers to use procedural generation of narration chains, analyze and test the designed game plots. An additional goal of this approach strongly focused on using the JSON data format to describe actions that can be taken in world states is the impact of JSON data structures on the application of graph rules, which may be important in the problems of artificial intelligence.

CCS CONCEPTS

• **Human-centered computing**; • **User centered design**; • **Human computer interaction (HCI)**; • **Software and its engineering**; • **Interactive games.**;

KEYWORDS

Decision Support, Collaborative Design, Graph Transformations, Procedural Storytelling

ACM Reference Format:
Iwona Grabska-Gradzińska, Leszek Nowak, Wojciech Palacz, and Ewa Grabska. 2021. Application of Graphs for Story Generation in Video Games.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACSW '21, February 01–05, 2021, Dunedin, New Zealand
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8956-3/21/02…$15.00
https://doi.org/10.1145/3437378.3442693

In *2021 Australasian Computer Science Week Multiconference (ACSW '21), February 01–05, 2021, Dunedin, New Zealand*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3437378.3442693

1 INTRODUCTION

Plot representation is a difficult concept in non-linear video games as the player is unpredictable and often creates unexpected situations. In movies there are screenplays, story boards, scripts. In games, where the story is branching, dynamic and often takes multiple hours, it is essential to have dedicated tools for the creation and maintenance of branching narration and plots.

Trying to address the problem of branching storytelling of video games, various tools are used. Unfortunately, those tools are designed for specific games and are rarely shared by the development companies. Only sometimes we get a glimpse of how this is done by a specific company [1].

We propose a method of handling collaborative story design and world building that will help develop plot arcs, test them in a context of a video game, or even procedurally generate adventures.

2 RELATED WORK

The concept of computer aided design using graph models is widely used in industrial applications [2, 3] as well as in game design studies [4-8]. Especially the layered graphs are utilized for real world system modelling [9, 10]. The mainstream game production industry employs graphs for quest generation [9] although this is not well documented in literature as companies tend not to share their inhouse tools and documentation. In our work we develop such tool and present its applications and results achieved thus far [11-13].

Procedural generation is a concept that is getting more elaborate and is becoming more common in video games [14-17]. It allows to adjust the system to current needs [18, 19]. It usually takes the form of terrain generation relying strongly on a Perlin noise algorithm or similar. Procedural story generation, on the other hand, is less common and quite difficult to do appropriately. Below is a fragment of a story, generated by an online tool [20]. The fragment, although unusual, could be considered somewhat funny and acceptable:

> "«Look John», growled Amanda, with an annoying glare that reminded John of sad bears. «It's not that I

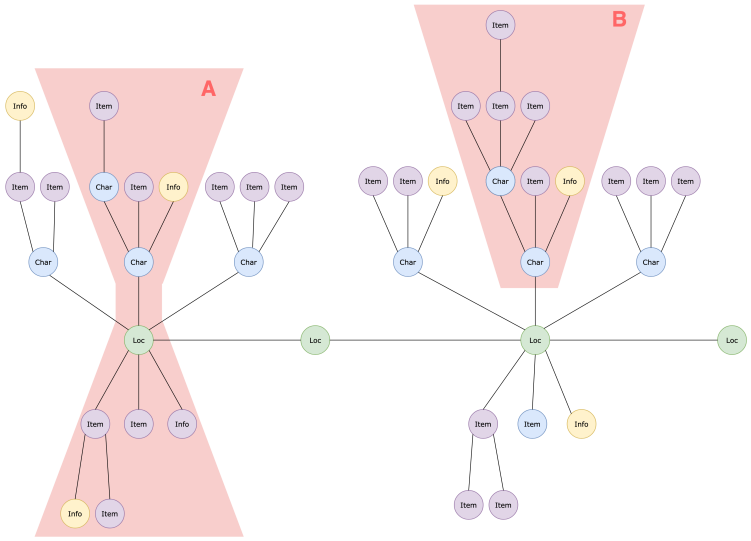


Figure 1: World State Graph. Green nodes "Loc" stand for locations, blue ones "Char" stand for characters, violet ones "Item" stand for items and yellow ones "Info" stand for narrative information. The red shape "A" highlights a sheaf subgraph and "B" shows a half-sheaf subgraph.

> don't love you, but I want loads of money. You owe me 776 euros.»"

Next is a fragment from the same story but far more unexpected:

> "John looked back, even more broken and still fingering the warm teapot. «*Amanda, I'm pregnant*», he replied."

The given example was made using prewritten word structures, where specific words or phrases are replaced using random or user generated input [20].

Tools for procedural story generation can take various forms, with anything from AI and deep learning to simple templates with randomized context. One especially interesting tool striking midground is Tracery – an author-focused generative text tool where rules and constrains are defined much like in programming, but for a story, not for a computer program [21, 22].

A prominent example of story generation in video games is in a game "Caves of Qud" where whole books are generated. While most of these books has no practical value, some can provide context about the lore of the game while others provide important in-game knowledge, e.g., information about diseases and appropriate treatments [23, 24].

Probably the most elaborate procedural world generation and story building is done in the game "Dwarf Fortress" by Tarn and Zach Adams. Here the procedure involves generation of an entire world, from a life story of a single character to a history of entire civilizations over hundreds of years, all written down and for the player to discover [25, 26].

3 GRAPH MODEL OF NARRATIVE ELEMENTS

The graph model of a game consists of two separate graphs: a graph model of the world state and a graph model of the gameplay.

3.1 World state graph

The graph model of the game world (the World State Graph) is layered and hierarchical. Every graph node stands for one object in the game world and belongs to one of four layers: layer of locations, layer of characters, layer of items or layer of narrative elements. Graph edges represent dependencies between objects: spatial dependencies, ownership, affiliation etc. Node attributes represent object properties. The layered graph used in the system is formally defined in [12].

Sheaf subgraph. The shape of the World State Graph is determined by the real-world dependencies among the elements of the game world. The fundamental layer is the locations layer, where every other element belongs to a specific location or belongs to a character present in a location. The initial element of the action taken in the game world is the character, placed in the specific location. Location with its belongings and character with his belongings can be visualized as a shape like a sheaf. The character with his belongings only can be visualized as a half-sheaf, as seen in .Figure 1

3.2 Game event as graph production

The events which change the game world can be easily represented as graph rules, also known as productions. A production is a pair of graphs called the left-hand side and the right-hand side of the production. Application of the production to a given World State Graph causes replacement of the nodes of the left-hand side by the nodes of the right-hand side and their embedment in the World State Graph.

The first step of applying the production is to match, i.e., to fit the left-hand side to the World State Graph. A successful fit means that there is a subgraph in the WSG which is equal to the left-hand side graph, as seen in Figure 2

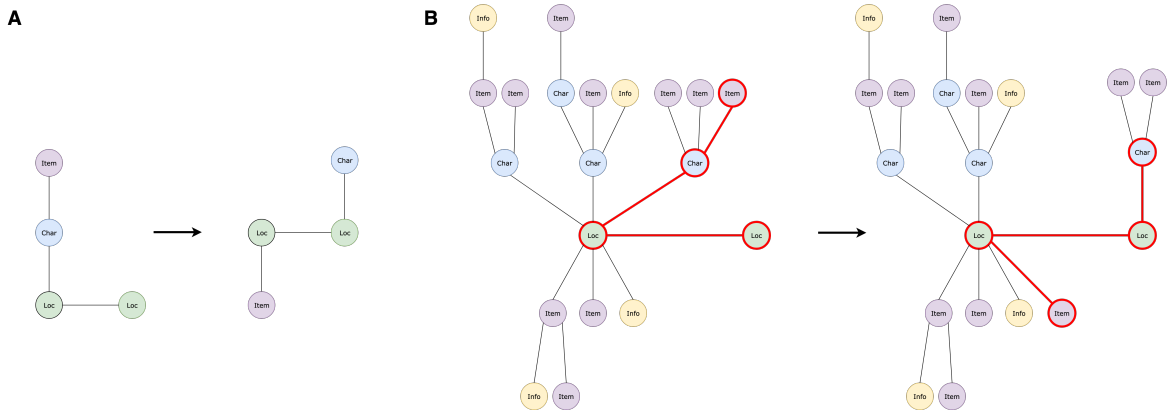


Figure 2: A – left and right side of the production. B – application of the production to the Word State Graph from fig. 1

3.3 Gameplay graph

Every defined production is identified by an ID. The sequence of the productions applied one after another represents the chain of player actions. The chain which starts with the initial production and ends with the production including wining condition node in the right-hand side graph represents the gameplay of one user. The gameplay model consists of all action chains of all players or, in the more general approach, all possible production chains.

4 IMPLEMENTATION, WORKFLOW AND ADVANTAGES

The presented approach strongly focuses on using the JSON data format to describe world state and actions that can be taken in certain situations / world states. We decided to use the JSON format as it is easy to read and can be imported into software in unchanged form as a dictionary-based data structure.

4.1 Data structure of the game world

In the proposed implementation we represent the game world as a dictionary of locations. Every location is also represented by a dictionary, as shown in the code snippet representing the "Road" location.

The data structure uses key-value representation and corresponds with a hierarchical graph where key represents a graph node. The set of locations create a dictionary of locations that we refer to as World State Graph.

4.2 Interacting with the game world

Interaction with the game world is done via the graph productions described earlier. Each production has similar data structure to the game world and is represented in a form of a dictionary and stored as a JSON file. Such productions can be generated via a text editor or via the dedicated editing tool. Figure 3 depicts an example trade action as seen in the tool.

Interacting with the game world is straightforward, as we can search the World State Graph hierarchically by keys using the left-hand side of the production as a template. By matching all the keys we find if a production is applicable in a current state of the game world. If yes, then the World State Graph is modified according to the right-hand side of the production, thus creating a new World State Graph.

4.3 Initial assumptions

In the proposed work we assume every generated story has the same initial starting point. The designers prepared a list of productions based on the game mechanics and the narrative outline. There is

CODE SNIPPET 1: the data structure of a typical location

```
"Road": {
        "Connections": { "Forest": { }, "Town of Honeywood": { }, "Farm": { }, "The lone tree": { } },
        "Characters": {
                "Princess": { "Items": { "Necklace": { "Attributes": { "value":"10" } }, "Coins": { "Attributes": { "quantity":
"7" } }    }
                },
                "Huntsman": { Items": { "Riffle": { } , "Ammo": "quantity": "5"} },
        },
        "Items": { "Rock": { "Attributes": { "heavy": { } } },
        "Attributes": { "Position": { "x": 12.2, "y": 145.4 } }
}
```

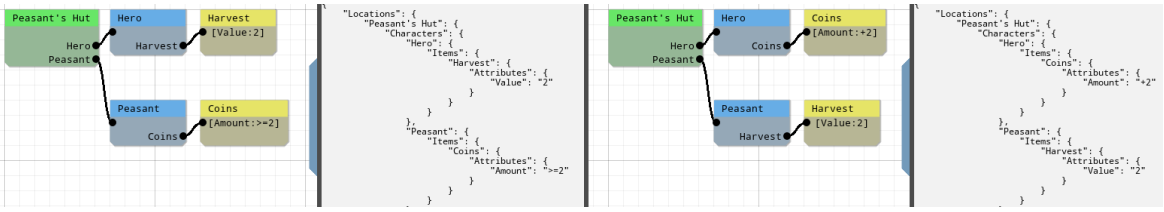


Figure 3: Left- and right-hand sides of the production represented as graph nodes and JSON files in the tool.

Table 1: Examples of the generic and derived detailed productions

Generic production	Example of derived detailed production
Fight (escape of the attacked one)	Throwing the troublemaker out of the inn
Theft	Theft of the poison from the wizard
Trade	Buying the poison from the wizard
Fight (attacker wins)	Fight with the dragon using elixir
Fight (attacker dies)	Fight with the dragon bare handed
Picking up an item	Getting the dragon tooth
Changing location	Coming from the road into the inn

one initial production and a set of final productions, which include defined winning or losing conditions. Applying any of them means the end of the game. The productions can be designed at different levels of abstraction (generic → detailed), to make the world as linear or open as needed.

4.4 Generic and detailed productions

A production can be designed for the specific World State Graph nodes, but also can be designed with the generic nodes "any item", "any location", "any character" (layer name as the only identifier of the node). It is very useful, especially with the common activities available for all characters in any circumstances, e.g. changing their location. These types of productions, which use the generic node types instead of the specific ones, are called "generic productions" as opposed to "detailed productions".

For the sake of coherence all the detailed productions are derived from the set of generic productions. It is useful in the design stage, but also fulfills its function during the production matching process. Table 1 contains examples of generic productions as well as detailed productions that are derived from them.

4.5 Dynamic set of productions available for given world graph

During the gameplay the set of currently available productions is generated on the base of a fitting function, which seeks a subgraph of the World State Graph identical to the left-hand side of the production. The possible actions are presented to the player (or the generative algorithm) as the list of options. After the player choses the action, the World State Graph changes accordingly and a new list is generated.

While preparing the set of the available actions, the list of both the generic and detailed productions is searched for fitting ones, but if there is a detailed production available, it will take precedence over its parent generic action. E.g., if there is a locked door in the player's path and he needs a key, a generic production would allow for passage without the necessary requirement (the key).

Player's entire gameplay can be described as a chain of chosen productions. First is the initial gameplay production (the beginning of the grand adventure) and the last is one of the final productions, i.e., these including winning or losing conditions. The number and succession of the productions is the result of a series of player decisions. The gameplay production chain can be analysed to check for player's decision preferences, play style, side quests taken, etc. The gameplay of two players can be compared as well.

Another situation is when the gameplay is to be automatically generated for the sake of testing. The final choice of applied decisions depends on the aim of testing. This can be random wandering in the game world, speed playing, testing for dead-ends in quests, finding gameplay loops, or in extreme cases a tree of all possible story chains can be generated (useful in mostly linear plots).

4.6 Expanding the set of possible production chains

The initial world state is created by designers and specifies the position and features of the main character. All the non-player characters (NPCs) can be placed in specific locations as well, but they also can be distributed randomly or even made autonomous with a set of actions of their own. The location of NPCs is important for the player only if he wants to perform an action involving an NPC. During gameplay the player creates the chain of actions by taking decisions (executing productions). Sometimes the chain of executed actions leads to the dead-end. One of the possible reasons is a design mistake, when the sequence of productions causes a clinch, regardless of player's and NPCs' decisions. Another reason can be a player's deliberate action, when he provokes the situation of a clinch on purpose.

Determined or undetermined world state. Procedural generation of production chains replaces player's decision process and helps to extend any possible chain of the player's decisions, especially to show all design clinches. The initially determined and constant world limits the available chains, although some number of chains would be possible to perform only if the NPCs were in specific locations. To eliminate this problem, we can change the way of matching productions. During the matching process we can take into consideration not the whole left-hand side of the production, but only the sheaf-subgraph of the main hero (indicated in red

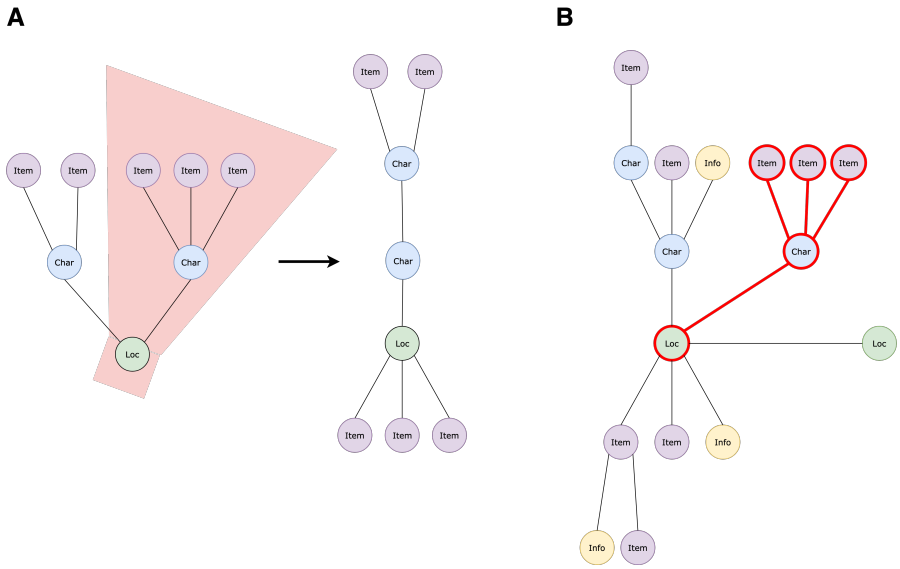


Figure 4: A – example of a production with indicated sheaf subgraph. B – a matching of this production to the World State Graph which includes only the indicated sheaf.

on Figure 4). In this approach we assume that all NPCs mentioned in the production can in some circumstances occupy any location, inasmuch as the story designer recognised the situation as acceptable, so that the production can be applied even if only the main hero's features match.

Loosened restrictions. This approach can be used during the gameplay to break a player's deadlock. As it is mentioned above, the existence of a detailed production blocks the usage of its parent generic production (e.g. buying a sword from the blacksmith precludes buying a sword from any other character). Removing this detailed production gives the player the possibility to buy the sword from any person who has a sword and can unblock the situation. This example shows that removing the detailed production, which is not applicable in the given circumstances, enables the use of more generic productions that otherwise are blocked by the detailed one.

Consistency of quest design. One of the benefits of having a production set that defines possible actions is the ability to dynamically swap, add or remove said actions. By being able to do so, it is possible to influence gameplay options in various ways. For example, consistency of quest lines can be validated and enforced if there is a loose end in a quest line. A practical example of such a situation would be a quest where the player has to obtain a specific item (e.g. a mystery box). Normally the quest will have a way to obtain such an item (e.g. bartering with a non-player character will provide the item). If for any reason the player cannot acquire the item via barter, the quest should provide an appropriate resolution of such a situation. Unfortunately, in quest design quite often it is not possible to predict and prepare for every situation that the player can create. That is why in video games story designers make sure to mark important non-player characters as story characters, thus making them immune (usually immortal) to most player actions [27]. The problem is not limited to characters and can easily concern items or even narration. Hence the concept of a "Quest Item", an item that is immune to most of the actions (trading, dropping, etc.) that can involve items.

We can ensure quest consistency in the testing step where story arcs are validated. Alternatively, we can inject extra productions if the player purposefully created a clinch situation, e.g. eliminated a story character. Those injected productions can include a way to bypass the clinch, or simply undo the player action (e.g. the character is brought back to life).

5 CONCLUSION AND FUTURE WORK

The presented work is in constant development and so far, we managed to successfully collaborate in building worlds where multiple writers worked independently to create characters and locations. Furthermore, collaboration with artists allowed us to create premade animation sets for generic actions, and by doing so we are able to generate story animations illustrating how the plot played out. Those animations take the form of short scenes stitched together to form a mockup animation.

In future the story generation will be based on production chains and will be generated as plain text using Tracery library. We are exploring options to employ an evolutionary algorithm for dynamic production generation as it will generate unexpected situations that surely will be unpredictable.

REFERENCES

[1] Mike Cosimano. 2017. CD Projekt Red designer breaks down The Witcher 3's best quest. Retrieved November 9, 2020 from https://www.destructoid.com/stories/cd-projekt-red-designer-breaks-down-the-witcher-3-s-best-quest-438115.phtml
[2] Grażyna Ślusarczyk. 2018. Graph-based representation of design properties in creating building floorplans. Computer Aided Design 95, (February 2018), 24–39. https://doi.org/10.1016/j.cad.2017.09.004
[3] Barbara Strug, Anna Paszyńska, Maciej Paszyński, and Ewa Grabska. 2013. Using a graph grammar system in the finite element method. International Journal of Applied Mathematics and Computer Science 23, 4 (December 2013), 839–853. https://doi.org/10.2478/amcs-2013-0063

[4] Daniel Sanchez and Hector Florez. 2018. Improving Game Modeling for the Quoridor Game State Using Graph Databases. In Proceedings of the International Conference on Information Technology & Systems (ICITS 2018). ICITS 2018. Advances in Intelligent Systems and Computing, Vol. 721, pp. 333–342. Springer, Cham. https://doi.org/10.1007/978-3-319-73450-7_32

[5] Johannes Konert, Viktor Wendel, Stefan Göbel, and Ralf Steinmetz. 2012. Towards an analysis of cooperative learning-behaviour in social dilemma games. In Proceedings of the European Conference on Games-based Learning (ECGBL 2011), pp. 329–332

[6] Johannes Putzke, Kai Fischbach, and Detlef Schoder. 2010. Power structure and the evolution of social networks in massively multiplayer online games. In 18th European Conference on Information Systems (ECIS 2010), 13 pages.

[7] Masafumia Sato, Koichib Anada, and Masayoshia Tsutsumi. 2017. Formulations of patterns by a graph model for the game of Go. Journal of Computational Methods in Sciences and Engineering, 17, S1, (January 2017) S111–S121. https://doi.org/10.3233/JCM-160684

[8] Imran Khaliq and Zachary Watson. 2018. The Omni Framework: A Destiny-Driven Solution to Dynamic Quest Generation in Games. 2018 IEEE Games, Entertainment, Media Conference (GEM), Galway, 2018, pp. 306-311. https://doi.org/10.1109/GEM.2018.8516526.

[9] Bhadrachalam Chitturi, Srijith Balachander, Sandeep Satheesh, and Krithic Puthiyoppil. 2018. Layered Graphs: Applications and Algorithms. Algorithms, 11, 7, (June 2018) pp. 93. https://doi.org/10.3390/a11070093

[10] Bhadrachalam Chitturi and T Srinath. 2018. Hard Problems on Layered Graphs: Parallel Algorithms and Improvements, 2018 IEEE 8th International Advance Computing Conference (IACC), Greater Noida, India, pp. 207-212, https://doi.org/10.1109/IADCC.2018.8692129.

[11] Iwona Grabska-Gradzińska, Leszek Nowak, Wojciech Palacz, and Ewa Grabska. 2020. Collaborative Story Generation using JSON in Godot Engine as Decision Making Support Tool. In International Conference on Cognitive Decision Support Systems & Technologies Proceedings (ICDSST 2020). 27-29 May 2020, University of Zaragoza,Zaragoza, Spain, 4 pages

[12] Iwona Grabska-Gradzińska, Leszek Nowak, Wojciech Palacz, and Ewa Grabska. 2020. Automatic Story Generation Based on Graph Model Using Godot Engine. In Artificial Intelligence and Soft Computing (ICAISC 2020). Lecture Notes in Computer Science, Vol. 12415. Springer, Cham. https://doi.org/10.1007/978-3-030-61401-0_37

[13] Iwona Grabska-Gradzińska, Leszek Nowak, Wojciech Palacz, and Ewa Grabska. 2020. Towards Automatic Generation of Storyline Aided by Collaborative Creative Design. In Cooperative Design, Visualization, and Engineering (CDVE 2020). Lecture Notes in Computer Science, Vol. 12341. Springer, Cham. https://doi.org/10.1007/978-3-030-60816-3_6

[14] Jonathon Doran and Ian Parberry. 2011. A prototype quest generator based on a structural analysis of quests from four MMORPGs. In Proceedings of the 2nd International Workshop on Procedural Content Generation in Games (PCGames '11). Association for Computing Machinery, New York, NY, USA, Article 1, 1–8. https://doi.org/10.1145/2000919.2000920

[15] Rubén H. García-Ortega, Pablo García-Sánchez, Juan J. Merelo, María Isabel G. Arenas, Pedro A. Castillo, and Antonio M. Mora. 2015. How the World Was MADE: Parametrization of Evolved Agent-Based Models for Backstory Generation. In Applications of Evolutionary Computation (EvoApplications 2015). Lecture Notes in Computer Science, Vol. 9028. Springer, Cham. https://doi.org/10.1007/978-3-319-16549-3_36

[16] Rubén H. García-Ortega, Pablo García-Sánchez, Juan J. Merelo, Aránzazu San-Ginés, and Ángel Fernández-Cabezas. 2016. The Story of Their Lives: Massive Procedural Generation of Heroes' Journeys Using Evolved Agent-Based Models and Logical Reasoning. In Applications of Evolutionary Computation (EvoApplications 2016). Lecture Notes in Computer Science, Vol 9597. Springer, Cham. https://doi.org/10.1007/978-3-319-31204-0_39

[17] Matheus R. F. Mendonça and Artur Ziviani. 2018. Network-Based Procedural Story Generation. Comput. Entertain. 16, 3, Article 3 (September 2018), 18 pages. https://doi.org/10.1145/3236494

[18] James Pita, Brian Magerko, and Scott Brodie. 2007. True story: dynamically generated, contextually linked quests in persistent systems. In Proceedings of the 2007 conference on Future Play (Future Play '07). Association for Computing Machinery, New York, NY, USA, 145–151. https://doi.org/10.1145/1328202.1328228

[19] Daniel Brogaard Buss, Morten Vestergaard Eland, Rasmus Lystlund, and Paolo Burelli. 2015. The Quality System – An Attempt to Increase Cohesiveness Between Quest Givers and Quest Types. In Interactive Storytelling (ICIDS 2015). Lecture Notes in Computer Science, Vol 9445. Springer, Cham. https://doi.org/10.1007/978-3-319-27036-4_41

[20] Plot Generator. Set of text generator tools (January 2019) Retrieved November 9, 2020 from https://www.plot-generator.org.uk/

[21] Kate Compton, Benjamin Filstrup, and Michael Mateas. 2014. Tracery: Approachable story grammar authoring for casual users. In Papers from Seventh Intelligent Narrative Technologies Workshop (INT 07), pp. 63–67.

[22] Kate Compton, Ben Kybartas, and Michael Mateas. 2015. Tracery: an author-focused generative text tool. In Interactive Storytelling (ICIDS 2015). Lecture Notes in Computer Science, Vol 9445. Springer, Cham. https://doi.org/10.1007/978-3-319-27036-4_14

[23] Jason Grinblat and C. Brian Bucklew. 2017. Subverting historical cause & effect: generation of mythic biographies in Caves of Qud. In Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG '17). Association for Computing Machinery, New York, NY, USA, Article 76, 1–7. https://doi.org/10.1145/3102071.3110574

[24] Jason Grinblat and C. Brian Bucklew. 2020. Warm Rocks for Cold Lizards: Generating Meaningful Quests in Caves of Qud. In Experimental AI in Games Workshop (AIIDE 2020).

[25] Tarn Adams. 2017. Secret identities in dwarf fortress. In Proceedings Experimental AI in Games (AIIDE 2017), pages 22–25.

[26] Tarn Adams. 2015. Simulation principles from Dwarf Fortress. In Game AI Pro 2: Collected Wisdom of Game AI Professionals, pages 519–521. CRC Press, 2015.

[27] Anders Hagström. 2017. Poetically Man Dwells in Game Space: A Phenomenological Investigation of Video Games as Art. Ph.D. Dissertation. Uppsala University, Disciplinary Domain of Humanities and Social Sciences, Faculty of Arts, Department of Game Design. Retrieved from http://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-326132